CS2323 Computer Architecture

Project Scope Document

Autumn 2026

Working title

Integrated Computer Architecture Design, Simulation and Analysis Platform

A unified educational environment connecting source code, instruction sets, processor design, execution, architectural analysis, RTL simulation and virtual embedded hardware.

Proposed scope • Functional requirements • Milestones • Validation • Risks

1. Project Overview

1.1 Problem Statement

Computer architecture education involves several related activities: writing assembly code, understanding instruction encoding, studying processor datapaths, designing instruction sets, simulating processor execution, analysing memory hierarchies, and experimenting with hardware. These activities are often performed using separate tools, making it difficult to understand the complete relationship between source code, machine instructions, processor architecture, hardware execution and performance.

1.2 Proposed Solution

Develop an integrated, web-based Computer Architecture laboratory that connects these activities through a shared execution and artifact model. Users will be able to write code, compile or assemble it for supported architectures, design custom instruction sets, construct processor datapaths, execute programs, inspect registers and memory, explore caches and branch prediction, and experiment with generated RTL and virtual hardware.

The platform includes a Custom ISA Designer, Instruction Encoder/Decoder, Processor Design Studio, Compiler and Cross-Compilation Studio, Assembly-to-C decompilation, Memory Hierarchy and Cache Lab, Branch Prediction Lab, Beginner Code Editor and Memory Explorer, FPGA/RTL Sandbox, Virtual Embedded Board Studio, and Persistent Workspace.

2. Objectives

- Provide a unified environment for learning and experimenting with computer architecture.
- Support standard RISC-V assembly and machine-code execution.
- Allow users to define and experiment with custom instruction sets.
- Provide visual processor construction using reusable hardware components.
- Connect compilation, assembly, instruction encoding and execution.
- Visualize execution through synchronized code views and control-flow graphs.
- Inspect registers, memory, instruction execution and processor state.
- Demonstrate cache behaviour, branch prediction, pipeline hazards and execution costs.
- Provide an isolated environment for RTL simulation and supported FPGA workflows.
- Support interactive embedded-system experiments through virtual boards.
- Provide reproducible experiments and measurable architectural results.

3. Functional Scope

3.1 Custom ISA Designer

Must-have functionality

- Define an instruction set with configurable instruction formats.
- Configure opcode fields, register fields, immediate fields and instruction widths.
- Define architectural registers, register widths and aliases.
- Define instruction semantics and supported addressing modes.
- Validate instruction formats and detect conflicting encodings.
- Generate instruction encoders and decoders from ISA definitions.
- Support assembly syntax and machine-code generation for supported custom ISAs.
- Save, load and version ISA definitions; provide reference execution for supported instructions.

Stretch functionality

- Advanced instruction formats and extensions.
- More sophisticated assembler diagnostics and optimization.
- Automatic generation of compiler-backend components.
- Broader support for floating-point, vector or specialized instructions.

Acceptance criteria

A user can define a small working ISA, encode/decode its instructions, assemble a test program, execute it through the reference engine and verify the resulting architectural state.

3.2 Instruction Encoder and Decoder

Must-have functionality

- Convert assembly instructions into binary and hexadecimal machine code.
- Decode machine code into instruction fields and assembly representations.
- Display field boundaries, register operands, opcode values and immediate values.
- Support standard RISC-V instructions implemented by the platform.
- Dynamically support instruction formats defined in the Custom ISA Designer.
- Report invalid instructions and malformed encodings.

Stretch functionality

- Bit-level editing and immediate-field visualization.
- Side-by-side comparison of encodings across supported ISAs.
- Integration with compiler output and processor execution traces.

Acceptance criteria

Supported instructions can be encoded and decoded consistently, with round-trip tests where the instruction representation permits them.

3.3 Processor Design Studio

Must-have functionality

- Visual editor for constructing processor datapaths.
- Reusable components: program counters, instruction memory, decoders, register files, ALUs, multiplexers, control units, data memory and branch units.
- Wires, connections, input/output ports and signal widths.
- Reusable hierarchical components containing other components.
- Combinational and sequential components with defined execution semantics.
- Validate connectivity, widths and incomplete designs.
- Persist designs in a structured architecture model and associate each design with its ISA.
- Execute a defined subset of processor designs and inspect processor state.

Stretch functionality

- Advanced multicycle and pipelined processor configurations.
- Forwarding networks, stall logic and hazard detection.
- Automated RTL generation for a larger subset of designs.
- Parameterized components, reusable component libraries and advanced timing/resource estimates.

Acceptance criteria

A supported processor design executes a test program, updates architectural state correctly and exposes internal execution state for inspection.

3.4 Compiler and Cross-Compilation Studio

Must-have functionality

- Code editor for supported source languages and assembly.
- Compile supported C/C++ programs using available toolchains.
- Support available targets such as x86-64, AArch64 and RISC-V.
- Select compiler and optimization flags where supported.
- Display generated assembly and machine code.
- Execute supported output using the appropriate reference simulator or compatible processor.
- Use saved custom ISAs as targets where a compatible backend exists.
- Save source files, compiler configurations and generated artifacts.

Assembly-to-C translation and synchronized visualization

- Translate supported assembly into equivalent C-like code through decompilation/reconstruction.
- Display original assembly and translated code side by side.
- Display a control-flow graph with basic blocks, branches and loops.
- Associate executed instructions with source lines, translated statements and graph nodes where mappings are available.
- Highlight the current instruction and corresponding graph node during execution.
- Update register and memory views as execution progresses.

Stretch functionality

- Broader language support and additional target architectures.
- Dedicated LLVM backend for custom ISAs.
- More advanced optimization comparisons.
- Improved reconstruction of high-level control structures and data types.
- Register allocation and intermediate-representation visualizations.

Technical constraint

Defining a custom ISA does not automatically make an existing compiler support it. Custom ISA compilation requires a compatible compiler backend or a restricted-language compiler. Assembly-to-C translation may not recover original source; label output as reconstructed C-like code and validate behaviour rather than textual similarity.

Acceptance criteria

Supported programs can be compiled or assembled, inspected and executed. The interface synchronizes code views and the execution graph when reliable mappings are available.

3.5 Beginner Code Editor and Memory Explorer

Must-have functionality

- Accessible editor for beginner assembly exercises and supported source languages.
- Show register values and architectural state.
- Display text, data, BSS, stack and heap regions where supported.
- Inspect memory addresses, stored values, pointers and byte representations.
- Provide hexadecimal, binary and decimal views.
- Provide stepping, reset, breakpoints and execution controls where supported.

- Show program counter, stack pointer and memory changes.

Stretch functionality

- Richer variable-level debugging and stack-frame visualization.
- Architecture-specific memory-layout views.
- Advanced watch expressions and memory-access history.

Acceptance criteria

A student can execute a supported program step by step and explain how its registers and memory change.

3.6 Memory Hierarchy and Cache Lab

Must-have functionality

- Model a configurable cache using access traces.
- Support implemented direct-mapped, set-associative and fully associative configurations.
- Configure capacity, line size and associativity.
- Demonstrate tag/index/offset address decomposition.
- Show hits, misses, evictions and replacement decisions.
- Report hit rate, miss rate and simulated memory-access costs.
- Compare configurations using the same workload.

Stretch functionality

- Additional replacement and write policies.
- Multilevel cache hierarchies.
- Interactive cache-state timelines.
- Direct integration with processor traces.

Acceptance criteria

Cache outcomes match deterministic reference tests and reported metrics are reproducible.

3.7 Branch Prediction Lab

Must-have functionality

- Trace-driven branch-prediction experiments.
- Static prediction schemes required by the project.
- Display predicted and actual outcomes.
- Count correct predictions and mispredictions.
- Visualize prediction state and branch outcomes.
- Report prediction accuracy and misprediction rate.
- Show configured misprediction penalties when the timing model supports them.

Stretch functionality

- One-bit and two-bit dynamic predictors.
- Local/global history and tournament predictors.
- Pipeline-flush visualization.
- Integration with actual processor execution traces.

Acceptance criteria

Predictor results match hand-verified traces, and comparisons use identical workloads and documented assumptions.

3.8 FPGA and RTL Sandbox

Must-have functionality

- Sandbox for supported Verilog/SystemVerilog designs.
- RTL simulation using an appropriate HDL simulation tool.
- Run testbenches and display results.
- Waveform inspection for supported designs.
- Isolation of untrusted source code and simulation workloads.
- RTL generation from the defined subset of processor designs where feasible.

Stretch functionality

- Expanded synthesizable RTL generation.
- Synthesis reports and resource estimates.
- Timing analysis.
- Programming a physical FPGA board, subject to board availability and toolchain access.

Acceptance criteria

A supported RTL design passes a testbench in the isolated sandbox, and the user can inspect its simulation results.

3.9 Virtual Embedded Board Studio

Must-have functionality

- Virtual board environment for supported embedded targets.
- Code editor alongside a board visualization.
- Simulate selected peripherals such as LEDs, buttons, switches, timers and UART.
- Run, pause, reset and inspect supported firmware.
- Display peripheral state and relevant execution output.
- Provide one complete demonstration connecting code execution to visible hardware behaviour.

Stretch functionality

- Additional board models such as ESP32, Raspberry Pi Pico, STM32 or a virtual RISC-V board.
- Displays, sensors, motors and communication interfaces.
- More realistic board-specific simulation.
- Student-custom processor boards.

Acceptance criteria

Supported firmware produces expected simulated peripheral behaviour and can be validated automatically.

3.10 Persistent Workspace and Project Management

Must-have functionality

- Save and load user projects.
- Persist source code, ISA definitions, processor designs and experiment configurations.
- Reuse ISA definitions across compatible modules.
- Associate processor designs with their implemented ISA.
- Store compiled artifacts, execution results and relevant metadata.
- Maintain enough version information to reproduce experiments.

Stretch functionality

- Registration and authentication.
- Project sharing.
- Instructor-managed courses and assignments.
- Submission history, automated grading and course templates.

Acceptance criteria

A saved project can be reopened without losing its supported artifacts or configuration relationships.

4. Cross-Module Integration

Integration is a core requirement, not merely a final presentation feature. The platform should demonstrate this workflow:

- Define or select an ISA.
- Encode or assemble a program for that ISA.
- Compile a supported source program when a compatible compiler backend exists.
- Select or construct a processor that implements the ISA.
- Execute using the appropriate engine.
- Inspect registers, memory and instruction behaviour.
- Inspect execution through synchronized code views and control-flow graphs.
- Analyse cache behaviour and branch prediction using supported traces.
- Compare simulated architectural metrics.
- Save the experiment and reproduce its results.

The platform distinguishes three execution models: (1) reference ISA simulator, (2) processor simulator executing implemented datapath/control behaviour, and (3) RTL simulator executing Verilog/SystemVerilog. These are not interchangeable. A shared artifact model and trace format should include instruction address/identity, cycle index, register effects, memory accesses, branch outcomes and supported pipeline events.

5. Performance Measurement and Validation

Performance results describe simulated architectural behaviour unless explicitly measured on physical hardware.

- Static instruction count and dynamic instruction count.
- Simulated clock cycles, CPI and IPC.
- Pipeline stalls, flushes and branch mispredictions.
- Cache accesses, hits, misses, evictions and simulated memory-stall cycles.
- Instruction mix and generated code size.
- Hardware resource and timing reports when synthesis is performed.

Definitions: CPI = simulated cycles / dynamic instructions. IPC = dynamic instructions / simulated cycles.

Comparisons must use the same program, input, ISA semantics and documented simulation assumptions. Identify whether results came from a reference simulator, processor model, RTL simulation or actual FPGA measurement.

Validation strategy

- Unit tests for individual components and instruction encoding/decoding.
- ISA execution and processor-state tests.
- Compiler and assembler integration tests.
- Cache and branch-prediction reference tests.

- RTL testbench validation and board-behaviour tests for supported peripherals.
- End-to-end regression tests and reproducible performance experiments.

6. Must-Haves and Stretch Goals

All major modules described in this document remain part of the intended project scope. The initial implementation may support a defined subset of architectures, instructions, hardware components and boards, but each core module must have meaningful functionality and a testable demonstration.

Non-negotiable core:

- Custom ISA design and instruction encoder/decoder.
- Processor Design Studio.
- Compiler/cross-compilation and assembly-to-C-like reconstruction with synchronized visualization.
- Beginner editor and memory explorer.
- Memory hierarchy/cache and branch-prediction visualization.
- FPGA/RTL sandbox.
- Virtual embedded-board environment.
- Persistent workspace and integrated execution workflow.

Stretch goals: broad custom-ISA compiler support through LLVM; advanced dynamic predictors; complex pipeline configurations; advanced cache hierarchies; expanded boards/peripherals; physical FPGA deployment where possible; advanced synthesis/timing analysis; collaboration, instructor dashboards and assignment management.

Stretch classifications are provisional and should be reviewed against the course rubric and hardware availability.

7. Proposed Timeline

The exact deadline and team capacity are not yet confirmed. The following is a relative 12-week framework to adjust to the actual semester duration.

Phase	Weeks	Key activities	Exit criteria
1. Architecture and foundations	1–2	Review requirements; define modules, schemas, interfaces, initial ISA, build/test strategy and ownership.	Architecture and interface documents; repository and baseline tests.
2. Core implementation	3–6	Build initial ISA tools, processor design, compiler workflow, memory explorer, cache/prediction labs, RTL and board foundations, persistence.	Each core module has a working demo and automated tests.
3. Integration	7–9	Connect ISA, assembler, processor and traces; add synchronized views, metrics, RTL/board integration.	At least one end-to-end workflow with integration tests.
4. Validation and delivery	10–12	Regression tests, reference comparisons, reproducible experiments, security checks, documentation and demo.	Working integrated system, validation results and final documentation.

If the submission deadline is earlier, prioritize early integration and validation by building a thin end-to-end workflow early. This changes implementation sequencing, not the intended module scope.

8. Team Organization

Suggested workstreams (to assign according to actual team size):

- ISA, assembler and binary tooling.
- Processor design and simulation.
- Compiler, decompilation and code visualization.

- Cache, branch prediction and performance metrics.
- RTL sandbox and virtual hardware.
- Frontend, backend, persistence and integration.

Workstreams should agree on interfaces and artifact formats before implementing dependent modules independently.

9. Risks and Mitigations

Risk	Mitigation
Large combined scope	Parallel workstreams, staged depth and early integration.
Incompatible interfaces	Agree on shared schemas and API contracts early.
Custom ISA/compiler complexity	Separate ISA description, assembler, reference execution and compiler-backend responsibilities.
Incorrect hardware semantics	Define component semantics and validate against reference cases.
Misleading performance results	Document timing models; distinguish simulation from physical measurements.
Assembly-to-C reconstruction limits	Label output as reconstructed C-like code and validate behaviour.
RTL sandbox security	Isolate untrusted execution, disable unnecessary networking and enforce resource limits.
Limited FPGA access	Keep RTL simulation independent; make physical deployment conditional.
Late integration failures	Use continuous integration and end-to-end smoke tests.
Incomplete documentation	Update module documentation alongside implementation.

10. Assumptions and Open Decisions

- Final submission deadline and actual semester duration.
- Team size and module ownership.
- Access to FPGA boards and department infrastructure.
- Initial RISC-V variant and instruction subset.
- Supported compiler toolchains and cross-compilation targets.
- Which processor designs will be executable and eligible for RTL generation.
- Initial virtual-board targets and simulation backends.
- Authentication, hosting and compute-resource requirements.

The architecture should allow expansion without claiming that every architecture, compiler, processor design or board is supported from the beginning.

11. Final Deliverables

- Integrated web-based platform.
- Functional Custom ISA Designer and instruction encoder/decoder.
- Processor Design Studio with executable supported designs.
- Compiler and cross-compilation workflow.
- Assembly-to-C-like reconstruction and synchronized visualization.
- Register, memory and code-execution inspection.
- Cache and branch-prediction analysis.
- RTL simulation sandbox and virtual embedded-board demonstrations.

- Persistent project artifacts and configurations.
- Automated tests, regression tests and validation results.
- Reproducible performance comparisons.
- Architecture documentation, setup instructions, final technical report and demonstration.

12. Definition of Project Success

The project is successful when the core modules operate meaningfully, their supported behaviours are validated, and the integrated platform demonstrates the connection between software, instructions, processor architecture, execution state and architectural performance.

The strongest demonstration follows a program from source code or assembly through encoding and execution, inspects its effects on registers and memory, visualizes its control flow, analyses cache and branch behaviour, and connects supported processor designs to RTL simulation and virtual hardware.

The primary objective is to meet the technical and academic requirements of CS2323. Potential adoption by the instructor or future batches is a secondary benefit.